

Section-1: Foundational Concepts & History

1. Who is the founder of C++?

- A. Dennis Ritchie
- B. James Gosling
- C. Bjarne Stroustrup
- D. Anders Hejlsberg

Answer: C

2. C++ was first developed at which organization?

- A. Microsoft
- B. MIT
- C. Bell Labs
- D. IBM

Answer: C

3. What was the original name of C++?

- A. Super C
- B. Turbo C
- C. C Advance
- D. C with Classes

Answer: D

4. In which decade was C++ created?

- A. 1960s
- B. 1970s
- C. 1980s
- D. 1990s

Answer: C



Section-2: Visual Studio Project Creation

5. To create a new C++ project in Visual Studio, you select:

- A. Console App
- B. Static Library
- C. Empty Project
- D. Windows Application

Answer: C

6. What shortcut compiles a single file?

- A. Ctrl + F5
- B. Ctrl + Shift + B
- C. Ctrl + F7
- D. Alt + F7

Answer: C

7. What shortcut builds the entire solution?

- A. Ctrl + F7
- B. Ctrl + Shift + B
- C. Ctrl + F5
- D. Alt + B

Answer: B

8. What shortcut runs the application without debugging?

- A. F5
- B. Ctrl + F5
- C. Alt + F5

D. Shift + F5

Answer: B

Section-3: Compilation vs Linking

9. The compiler generates which file extension?

- A. .exe
- B. .obj
- C. .dll
- D. .lib

Answer: B

10. The linker generates which file?

- A. .asm
- B. .obj
- C. .exe
- D. .dll

Answer: C

11. Where is machine code stored before linking?

- A. .exe
- B. .obj
- C. .txt
- D. .cpp

Answer: B

12. Which stage resolves external symbol references?

- A. Compilation
- B. Assembling
- C. Linking
- D. Execution

Answer: C



Section-4: PE & ELF Formats

13. Windows executables use which format?

- A. ELF
- B. PE
- C. COFF
- D. MACH-O

Answer: B

14. Linux binaries follow which format?

- A. PE
- B. EXE
- C. ELF
- D. BIN

Answer: C

15. Can .exe compiled in Windows run on Linux directly?

- A. Yes
- B. No
- C. Only in Debug
- D. Only Static Builds

Answer: B

Section-5: endl vs \n

16. What does endl do apart from adding newline?

- A. Clears console

- B. Flushes buffer
- C. Formats text
- D. Removes whitespace

Answer: B

17. Which is slower generally?

- A. \n
- B. endl

Answer: B

18. Which statement is TRUE?

- A. \n always flushes buffer
- B. endl never flushes buffer
- C. endl inserts newline + flush
- D. \n inserts newline + flush

Answer: C

Section-6: Build Stages

Match stage & output:

19. Preprocessing output extension?

- A. .o
- B. .exe
- C. .i
- D. .obj

Answer: C

20. Compilation output?

- A. .i
- B. .exe
- C. .asm
- D. .dll

Answer: C

21. Assembler output?

- A. .obj
- B. .exe
- C. .cpp
- D. .lib

Answer: A



Section-7: Offsets & Relocation

22. Offsets exist because object files must be:

- A. Static
- B. Portable
- C. Relocatable
- D. Debuggable

Answer: C

23. Linker converts offsets into:

- A. Stack values
- B. Physical memory
- C. Virtual addresses
- D. Reserved memory

Answer: C

Section-8: Typedef

24. typedef creates:

- A. New variable
- B. Alias name
- C. Macro
- D. Keyword

Answer: B

25. typedef struct syntax allows what?

- A. Removing struct keyword
- B. Removing typedef
- C. Dynamic memory
- D. Inline struct

Answer: A

Section-9: Pointer Basics

26. Pointer stores:

- A. Value
- B. Address
- C. Function
- D. File

Answer: B

27. In 64-bit system pointer size is:

- A. 2 bytes
- B. 4 bytes
- C. 6 bytes
- D. 8 bytes

Answer: D

28. & operator gives:

- A. Value
- B. Address
- C. Pointer
- D. Reference copy

Answer: B

29. * operator gives:

- A. Address
- B. Pointer
- C. Value at address
- D. Function call

Answer: C



Section-10: Pointer Arithmetic

30. ptr+1 for int moves:

- A. 1 byte
- B. 2 bytes
- C. 4 bytes
- D. 8 bytes (64-bit int assumed?)

Answer: C (assuming 4-byte int)

31. Two pointers can be subtracted only if:

- A. same type
- B. different type
- C. constant pointer
- D. void pointer

Answer: A

Section-11: Call by Value vs Address

Code

```
void swap(int x,int y){  
    int t=x; x=y; y=t;  
}
```

32. Effect?

- A. Swaps caller vars
- B. Does nothing to caller
- C. Runtime error
- D. Compile error

Answer: B

Code

```
void swap(int* x,int* y){  
    int t=*x; *x=*y; *y=t;  
}
```

33. Effect?

- A. Does not swap
- B. Swaps caller variables
- C. Divides values
- D. Undefined

Answer: B

Section-12: Function Pointer

34. Function pointer syntax:

- A. `int *ptr(int)`
- B. `int (ptr)(int)`
- C. `int ptr(int)`
- D. `(*ptr)int(int)`

Answer: B

Section-13: Arrays & Pointers

35. Array name refers to:

- A. Copy of array
- B. First element address
- C. Stack address
- D. Heap

Answer: B

Section-14: Character Pointers

36. Which prints string?

- A. `puts(ptr)`
- B. `cout<<ptr`
- C. Both
- D. None

Answer: C

Section-15: DMA

37. malloc returns:

- A. `int*`
- B. `void*`
- C. `float*`

D. char*

Answer: B

38. Memory allocated using malloc must be freed using:

- A. delete
- B. dispose
- C. release
- D. free

Answer: D

Section-16: Preprocessor

39. Preprocessor directives begin with:

- A. &
- B. #
- C. \$
- D. !

Answer: B

40. Macro replaces text during:

- A. Linking
- B. Execution
- C. Preprocessing
- D. Compilation

Answer: C

41. #ifndef executes block when macro is:

- A. Defined
- B. Undefined

Answer: B

42. #pragma pack(1) means:

- A. Maximum padding
- B. No padding
- C. 4-byte align
- D. Disable compiler

Answer: B



Section-17: Storage Classes

43. auto variables stored in:

- A. Heap
- B. Stack
- C. Data segment
- D. BSS

Answer: B

44. static local variables default value:

- A. Garbage
- B. 1
- C. NULL
- D. 0

Answer: D

45. extern refers to variable defined:

- A. In same block
- B. In another file
- C. Local only
- D. Stack

Answer: B

46. register variable stored in:

- A. Disk
- B. Heap
- C. CPU register
- D. BSS

Answer: C

Section-18: Static Library

47. Static library extension in Windows:

- A. .dll
- B. .exe
- C. .lib
- D. .o

Answer: C

48. Static library is linked at:

- A. Runtime
- B. Compile time
- C. Execution time
- D. Load time

Answer: B

49. Static library code becomes part of:

- A. DLL
- B. Kernel
- C. Final EXE
- D. Object file

Answer: C

Section-19: DLL Concepts

50. DLL stands for:

- A. Dynamic Load Library
- B. Dynamic Link Library
- C. Direct Link Library
- D. Data Link Layer

Answer: B

51. __declspec(dllexport) is used in:

- A. Client
- B. DLL Developer

Answer: B

52. __declspec(dllimport) used in:

- A. DLL Developer
- B. Client

Answer: B

53. extern "C" prevents:

- A. Debugging
- B. Name mangling
- C. Compilation
- D. Optimization

Answer: B

Section-20: DLL Errors

54. If DLL not copied to Debug folder:

- A. Compile error

- B. Linker error
- C. Runtime error
- D. No error

Answer: C

Section-21: PE vs DLL vs LIB

55. devpro.lib in DLL context is:

- A. Full library
- B. Import library
- C. Config file
- D. Manifest

Answer: B

Section-22: Memory Segments

56. BSS stores:

- A. Code
- B. Initialized static
- C. Uninitialized static/global
- D. Dynamic

Answer: C

57. Heap used for:

- A. Local variables
- B. Global
- C. Dynamic memory
- D. Code

Answer: C

58. Stack uses:

- A. FIFO
- B. LIFO
- C. FILO
- D. Random

Answer: B

Section-23: JNI & DLL

59. JNI is used for calling:

- A. Java from C++
- B. C/C++ from Java

Answer: B

60. JNI function name contains:

- A. Class name
- B. Package
- C. Method
- D. All above

Answer: D

CODE-BASED MCQs

61.

```
static int x;  
printf("%d",x);
```

Output?

- A. Garbage

- B. 0
- C. Error
- D. Undefined

Answer: B

62.

```
int *p;  
printf("%d", *p);
```

- A. Undefined behavior
- B. 0
- C. Garbage
- D. Crash always

Answer: A

63.

```
char str[]="hello";  
char *p=str;  
p++;  
printf("%c", *p);
```

Output:

- A. h
- B. e
- C. l
- D. o

Answer: C

64.

```
int arr[3]={10,20,30};  
int *p=arr;  
printf("%d", *(p+1));
```

- A. 10
- B. 20
- C. 30
- D. Compile error

Answer: B

65.

```
int x=10;  
int *y=&x;  
int **z=&y;  
printf("%d", **z);
```

Output?

- A. Address
- B. 10
- C. Error
- D. Garbage

Answer: B

66.

```
printf("%d", 5["abcde"]);
```

- A. Compile error
- B. e

- C. ASCII of e
- D. Runtime error

Answer: C

Because `5["abcde"] == *(5 + "abcde")`

67.

```
int x;  
printf("%d",x);
```

- A. 0
- B. Garbage
- C. Error
- D. Undefined always

Answer: B

68.

```
static int x;  
x=20;  
printf("%d",x);
```

- A. 0
- B. Garbage
- C. 20
- D. Error

Answer: C

69.

```
register int x=5;  
printf("%p",&x);
```

- A. Prints valid address
- B. Compile error may occur
- C. Always prints 0
- D. Runtime error

Answer: B

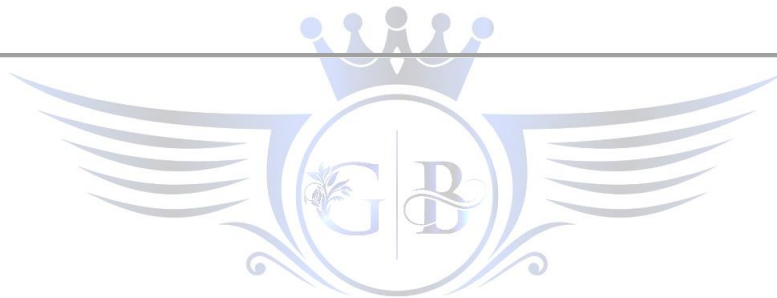
70.

```
void fun(){}  
int main(){  
    fun();  
}
```

Entry point is:

- A. fun
- B. main
- C. both
- D. none

Answer: B



Section-1: cin / cout vs scanf / printf

1. cin and cout belong to which classes?

- A. ostream and istream
- B. istream and ostream
- C. istream and stream
- D. cinclass and coutclass

Answer: B

2. Which operator is used by cin?

- A. <<
- B. >>
- C. ::
- D. ->

Answer: B

3. Which operator is used by cout?

- A. <<
- B. >>
- C. **
- D. ::

Answer: A

4. Which of the following is NOT required when using cin?

- A. &
- B. >>
- C. istream
- D. std namespace or scope

Answer: A

5. Which statement is TRUE?

- A. scanf requires format specifiers but cin does not
- B. cin requires format specifiers
- C. printf is type-safe
- D. cout requires %d and %s

Answer: A



Section-2: Clearing Input Buffer

6. Why does cin.getline() sometimes skip input?

- A. Because of memory leaks
- B. Because newline remains in input buffer
- C. Because getline is deprecated
- D. Because cin cannot take strings

Answer: B

7. What does the following do?

```
cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

- A. Flushes output buffer
- B. Clears keyboard buffer completely until newline
- C. Deletes previous input values
- D. Restarts input stream

Answer: B

Section-3: Function Declaration Requirement

8. In C++, calling a non-declared function causes:

- A. Warning only

- B. Runtime error
- C. Compile-time error
- D. Linker error

Answer: C

Section-4: Name Mangling

9. Name mangling exists to support:

- A. Dynamic binding
- B. Function overloading
- C. Exception handling
- D. Templates only

Answer: B

10. Which language DOES NOT mangle function names?

- A. C
- B. C++
- C. Java
- D. Python

Answer: A

Section-5: Dynamic Memory — new / delete

11. new operator allocates memory from:

- A. Stack
- B. Static memory
- C. Heap (free store)
- D. Register

Answer: C

12. delete[] must be used when memory is allocated using:

- A. malloc()
- B. new pointer
- C. new[]
- D. realloc()

Answer: C

13. Which one invokes constructors?

- A. malloc()
- B. calloc()
- C. new
- D. realloc()

Answer: C

14. Which one is an operator, not a function?

- A. malloc
- B. free
- C. new / delete
- D. calloc

Answer: C

Section-6: Scope Resolution (::)

15. The operator :: is used to access:

- A. Local members
- B. Global members hidden by locals
- C. Class objects
- D. Static objects

Answer: B

Given:

```
int val = 100;
int main(){
    int val = 20;
    cout << val;
    cout << ::val;
}
```

16. Output?

- A. 20 20
- B. 100 100
- C. 20 100
- D. Compile error

Answer: C

Section-7: Default Arguments

17. Default arguments must be specified:

- A. In function definition only
- B. In function declaration only
- C. Either
- D. Both

Answer: B

18. Which is valid?

- ```
void fun(int x=10,int y);
```
- A. Valid
  - B. Invalid

**Answer: B**

(Default parameters must trail)

---

### Section-8: Inline vs Macros

**19. Inline is a:**

- A. Compiler request
- B. Compiler command
- C. Macro
- D. Machine code directive

**Answer: A**

**20. A macro is processed during:**

- A. Linking
- B. Execution
- C. Preprocessing
- D. Compilation

**Answer: C**

**21. Which is type-safe?**

- A. Macro
- B. Inline function

**Answer: B**

**22. Inline functions may NOT be inlined when:**

- A. Recursive
- B. Too large
- C. Contain loops
- D. All above

**Answer: D**

---

## Section-9: Pointer to Constant / Constant Pointer

---

### Pointer to const

```
const int *p;
```

#### 23. Which is TRUE?

- A. p cannot change
- B. \*p cannot change
- C. Both cannot change
- D. Both can change

**Answer: B**

---

### Constant pointer

```
int * const p = &x;
```

#### 24. Which is TRUE?

- A. p cannot change
- B. \*p cannot change
- C. Both cannot change
- D. Both can change

**Answer: A**

---

### Const pointer to const

```
const int * const p=&x;
```

#### 25. Which is TRUE?

- A. p change allowed
- B. \*p change allowed
- C. Neither p nor \*p can change
- D. Both allowed

**Answer: C**

---

## Section-10: References

### 26. A reference must be:

- A. Initialized later
- B. Initialized at declaration
- C. Optional to initialize
- D. Dynamically assigned

**Answer: B**

### 27. A reference:

- A. Occupies memory like pointer
- B. Does not occupy separate memory
- C. Can be null
- D. Must be dereferenced manually

**Answer: B**

### 28. Reference vs Pointer — which is FALSE?

- A. Reference cannot be reseated
- B. Pointer can be reseated
- C. Reference auto-dereferences
- D. Reference can be incremented

**Answer: D**

---

### Code

```
int x=10;
int &r=x;
```

```
r=20;
cout<<x;
```

**29. Output?**

- A. 10
- B. 20
- C. Garbage
- D. Compile error

**Answer: B**

---

#### Section-11: Reference in Functions

```
void swap(int &a,int &b){
 int t=a; a=b; b=t;
}
```

**30. This demonstrates:**

- A. Call by value
- B. Call by address
- C. Call by reference
- D. Move semantics

**Answer: C**

---

#### Section-12: Ambiguity with References

**31. Why does this cause ambiguity?**

```
void disp(int);
void disp(int&);
int x=5;
disp(x);
```

- A. Both functions same signature after binding
- B. Compiler bug
- C. Runtime ambiguity
- D. Undefined behavior

**Answer: A**

---

#### Section-13: Memory Leak

```
void fun(){
 int *p=new int(5);
}
```

**32. What happens?**

- A. No issue
- B. Dangling pointer
- C. Memory leak
- D. Stack overflow

**Answer: C**

---

#### Section-14: Dangling Pointer

```
int *p=new int(5);
delete p;
```

**33. p is now:**

- A. Null
- B. Invalid address
- C. Still valid
- D. Reused automatically

**Answer: B**



### 34. Best practice?

- A. delete p
- B. p=NULLPTR
- C. Both
- D. None

**Answer: C**

---

### Section-15: Character Pointer Importance

#### 35. Why does this fail?

```
void disp(char* p);
disp("hello");
```

- A. String literal is const char\*
- B. disp not declared
- C. hello undefined
- D. cin conflict

**Answer: A**

---

### Section-16: Strings

#### 36. Which is INVALID?

- A. char s[]="abc";
- B. char s[10]="abc";
- C. char s[3]={'a','b','c'};
- D. char s[4]={'a','b','c'};

**Answer: C**

Because missing null terminator.

---

### getline vs cin

#### 37. getline reads input until:

- A. Tab
- B. Space
- C. Newline
- D. EOF only

**Answer: C**

---

### Section-17: Math Functions

#### 38. ceil(3.2) returns:

- A. 3
- B. 4
- C. 2
- D. 3.2

**Answer: B**

#### 39. trunc(3.9) returns:

- A. 3
- B. 4
- C. 3.9
- D. 2

**Answer: A**

---

### Section-18: Structures — C vs C++

#### 40. Major difference between struct and class in C++?

- A. struct supports functions, class does not
- B. struct defaults to public, class defaults to private

- C. class faster
- D. struct stack only

**Answer: B**

---

**41. Which operator accesses structure pointer members?**

- A. .
- B. ->
- C. \*
- D. ::

**Answer: B**

---

**42. Which is correct?**

Student s;  
Student \*p=&s;

- A. p.name
- B. s->name
- C. p->name
- D. name.p

**Answer: C**

---

**43. To free dynamically allocated array of structure:**

- A. delete
- B. free
- C. delete[]
- D. remove

**Answer: C**

---

**44. extern keyword means:**

- A. Allocate memory
- B. Prevent linkage
- C. Declare without defining
- D. Local variable

**Answer: C**

---

**45. If extern variable is declared but not defined:**

- A. Compile error
- B. Linker error
- C. Runtime error
- D. No error

**Answer: B**

---

## **Section-21: Linkage**

**46. static global variable has:**

- A. External linkage
- B. Internal linkage
- C. No linkage
- D. Local storage

**Answer: B**

---

**47. extern function has:**

- A. Internal linkage
- B. External linkage

- C. No linkage
- D. Runtime linkage

**Answer: B**

---

#### **Section-22: Type-Safe Linkage**

**48. C++ prevents mismatched function signatures using:**

- A. Macro guards
- B. Linker
- C. Name mangling
- D. Templates

**Answer: C**

---

#### **Section-23: Alternate Linkage**

**49. extern "C" is used to:**

- A. Speed execution
- B. Remove name mangling
- C. Force static binding
- D. Convert to C

**Answer: B**

---

**50. extern "C" is required when:**

- A. Calling C++ from Java
- B. Calling C from C++
- C. Calling Python from C++
- D. Calling DLL from OS

**Answer: B**

---

**51.**

```
int a=10;
int &r=a;
int b=20;
r=b;
cout<<a;
```

- A. 10
- B. 20
- C. Garbage
- D. Compile error

**Answer: B**

Reference refers to a, but assignment changes value of a.

---

**52.**

```
int x=10;
const int &r=x;
x++;
cout<<r;
```

- A. 10
- B. 11
- C. Error
- D. Garbage

**Answer: B**

---

**53.**

---

```
int x=5;
int * const p=&x;
int y=10;
p=&y;
A. Valid
B. Compile error
```

**Answer: B**

---

**54.**

```
const int *p;
int x=10;
p=&x;
*p=20;
A. Allowed
B. Compile error
```

**Answer: B**

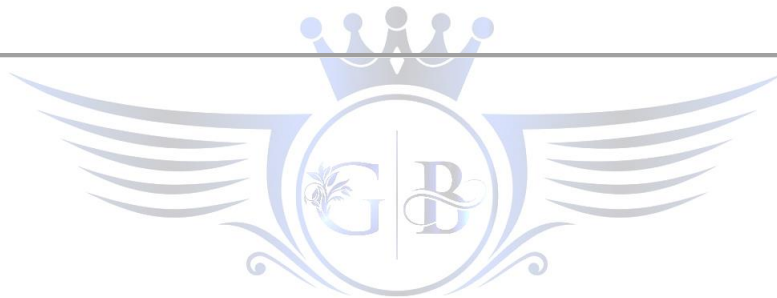
---

**55.**

```
const int x=10;
int *p=&x;
A. Valid
B. Invalid
```

**Answer: B**

---



**Section-1: Class & Object Basics**

**1. A class in C++ is:**

- A. A built-in type
- B. A user-defined data type
- C. A namespace
- D. A structure alias

**Answer: B**

**2. An object is:**

- A. A pointer
- B. A function
- C. A variable of a class
- D. A namespace member

**Answer: C**

**3. What is the size of an empty class object?**

- A. 0 bytes
- B. 2 bytes
- C. 4 bytes
- D. 1 byte

**Answer: D**

**4. Which statement is TRUE?**

- A. Class members are public by default
- B. Structure members are private by default
- C. Class members are private by default
- D. Both struct and class default to public

**Answer: C**

**5. Objects created using new are stored in:**

- A. Stack
- B. Heap
- C. Text segment
- D. Register

**Answer: B**

**6. What is required to destroy heap objects?**

- A. free()
- B. exit()
- C. delete
- D. Nothing

**Answer: C**

**Section-2: Member Functions & this Pointer**

**7. Functions defined inside the class are:**

- A. Always virtual
- B. Always static
- C. Always inline (requested)
- D. Never inline

**Answer: C**

---

**8. The 'this' pointer refers to:**

- A. Static class
- B. Global object
- C. Current invoking object
- D. Base class

**Answer: C**

---

**9. Which function CANNOT access this pointer?**

- A. Static member function
- B. Non-static function
- C. Virtual function
- D. Inline function

**Answer: A**

---

**Section-3: Constructors**

**10. Constructors:**

- A. Have return type void
- B. May return int
- C. Have no return type
- D. Must be static

**Answer: C**

---

**11. Which is NOT allowed for constructors?**

- A. Same name as class
- B. Overloading
- C. Being virtual
- D. Being inline

**Answer: C**

---

**12. When is default constructor provided automatically?**

- A. Always
- B. When no constructor is defined
- C. When only parameterized constructors exist
- D. When destructor exists

**Answer: B**

---

**13. When does compiler STILL generate copy constructor?**

- A. When user defines only default constructor
- B. When user defines only parameterized constructor
- C. In both A & B
- D. Never

**Answer: C**

---

**14. When does compiler NOT generate ANY constructor?**

- A. When copy constructor defined by user
- B. When destructor exists
- C. Always
- D. Never

**Answer: A**

---

**Section-4: Copy Constructor Invocation**

**15. Copy constructor is called when:**

- A. Passing object by value
- B. Returning object by value
- C. Initializing using another object
- D. All above

**Answer: D**

---

**16. Which line calls copy constructor?**

```
MyClass m1;
MyClass m2 = m1;
```

- A. Yes
- B. No

**Answer: A**

---

**17. Which line DOES NOT call copy constructor?**

```
MyClass m1, m2;
m2 = m1;
```

- A. Calls copy constructor
- B. Calls assignment operator
- C. Calls default constructor
- D. Causes error

**Answer: B**

---

#### **Section-5: Shallow vs Deep Copy**

**18. Default copy constructor performs:**

- A. Deep copy
- B. Shallow copy
- C. Move copy
- D. Bitwise compare

**Answer: B**

---

**19. Shallow copy is dangerous when class contains:**

- A. Static variables
- B. DMA / pointers
- C. Primitive types
- D. Inline functions

**Answer: B**

---

**20. In shallow copy, two objects share:**

- A. Independent memory
- B. Different addresses
- C. Same pointer memory
- D. Same constructor

**Answer: C**

---

#### **Section-6: Destructor**

**21. Destructor name is:**

- A. Classname()
- B. delete()
- C. ~Classname()
- D. destructor()

**Answer: C**

---

**22. Destructor cannot be:**

- A. Virtual
- B. Overloaded
- C. Inline
- D. Public

**Answer: B**

---

**23. Destructor executes automatically for:**

- A. Stack objects
- B. Heap objects
- C. Static objects only
- D. Namespace objects

**Answer: A**

---

**24. For heap objects, destructor executes when:**

- A. Program ends
- B. Memory full
- C. delete is called
- D. Object reference removed

**Answer: C**

---

#### **Section-7: Const Members & Const Functions**

**25. A const member must be initialized:**

- A. In constructor initializer list or at declaration
- B. Anywhere
- C. In destructor
- D. At runtime

**Answer: A**

---

**26. A const function:**

- A. Can modify all members
- B. Cannot modify any member
- C. Can modify members using this
- D. Must be inline

**Answer: B**

---

**27. Which can modify data inside const function?**

- A. const int
- B. static int
- C. mutable int
- D. reference

**Answer: C**

---

**28. A const object can call:**

- A. Only const functions
- B. Only static functions
- C. All functions
- D. No functions

**Answer: A**

---

#### **Section-8: Static Members**

**29. Static data members belong to:**

- A. Each object separately
- B. Class only
- C. Function
- D. Namespace

**Answer: B**

---

**30. Static data members must be defined:**

- A. Inside class always
- B. Outside class (pre-C++17)
- C. In constructor
- D. Cannot define

**Answer: B**

---

**31. Static member functions can access:**

- A. Only static members
- B. Only non-static members
- C. Both
- D. Global only

**Answer: A**

---

**32. Static member functions:**

- A. Have this pointer
- B. Do not have this pointer
- C. Always virtual
- D. Cannot be public

**Answer: B**

---

**33. static inline (C++17)**

- A. Must be defined outside
- B. Eliminates separate definition
- C. Not allowed
- D. Only for functions

**Answer: B**

---

## **Section-9: Singleton**

**34. Singleton constructor must be:**

- A. Public
- B. Protected
- C. Private
- D. Static

**Answer: C**

---

**35. Singleton prevents copying using:**

- A. Private copy constructor
- B. Static object
- C. Virtual destructor
- D. Const objects

**Answer: A**

---

**36. Singleton instance is returned via:**

- A. Non-static method

- B. Static method
- C. Friend function
- D. Destructor

**Answer: B**

---

#### **Section-10: this Pointer**

##### **37. this pointer stores:**

- A. Class address
- B. Object address
- C. Function address
- D. Heap address

**Answer: B**

---

##### **38. Which is equal?**

this->num

- A. \*num
- B. (\*this).num
- C. this.num
- D. this&num

**Answer: B**

---

#### **Section-11: Copy Constructor Reference Rule**

##### **39. Why must copy constructor accept parameter by reference?**

- A. Performance
- B. Avoid recursion (infinite calls)
- C. Prevent overload
- D. Restrict DMA

**Answer: B**

---

#### **Section-12: Stack vs Heap Objects**

##### **40. Destructor for stack objects:**

- A. Must call manually
- B. Called automatically
- C. Never called
- D. Called twice

**Answer: B**

---

##### **41. Destructor for heap object:**

- A. Automatic
- B. Manual using delete
- C. Depends
- D. Never called

**Answer: B**

---

#### **Section-13: Inline & Outside Definition**

##### **42. Function defined inside class is:**

- A. Private always
- B. Always inline (request)
- C. Always static
- D. Virtual

**Answer: B**

---

**43. Function defined outside class using :: is:**

- A. Always inline
- B. Never inline
- C. Can be inline with inline keyword
- D. Private only

**Answer: C**

---

#### **Section-14: DMA Array of Objects**

**44. Correct deletion of dynamic array of objects**

- A. delete ptr
- B. delete[] ptr
- C. free(ptr)
- D. remove(ptr)

**Answer: B**

---

**45. What happens if we use delete instead of delete[] ?**

- A. Compile error
- B. Runtime error always
- C. Only first object deleted
- D. All deleted anyway

**Answer: C**

---

#### **Section-15: True/False-Type MCQs**

**46. Even empty class has size 1 byte.**

- A. True
- B. False

**Answer: A**

---

**47. Constructors can be const.**

- A. True
- B. False

**Answer: B**

---

**48. Static functions can use this pointer.**

- A. True
- B. False

**Answer: B**

---

**49. Destructor can be overloaded.**

- A. True
- B. False

**Answer: B**

---

**50. Copy constructor is invoked when object is passed by value.**

- A. True
- B. False

**Answer: A**

---

**Section-1: Friend Functions & Friend Classes**

**1. A friend function is:**

- A. A member of a class
- B. A global function with special access to a class
- C. A static function
- D. A private function

**Answer: B**

---

**2. Where is a friend function declared?**

- A. Outside class only
- B. Inside class only
- C. Inside class but defined outside
- D. Inside namespace

**Answer: C**

---

**3. A friend function can access:**

- A. Only public members
- B. Public and protected only
- C. Only static members
- D. Private, protected, and public members

**Answer: D**

---

**4. A friend function is called using:**

- A. object.function()
- B. ClassName.function()
- C. Direct function call
- D. this pointer

**Answer: C**

---



**5. Friendship in C++ is:**

- A. One-way only
- B. Two-way only
- C. Always mutual
- D. Not transitive unless explicitly declared

**Answer: D**

---

**6. A friend class means:**

- A. All objects are friends
- B. Only specific object is friend
- C. All member functions of that class become friends
- D. None

**Answer: C**

---

**7. Which of the following can be declared as a friend?**

- A. Function
- B. Class
- C. Member function of another class
- D. All of the above

**Answer: D**

---

**8. A friend function requires:**

- A. Object to access private data
- B. No object ever
- C. Pointer always
- D. Static object

**Answer: A**

---

**9. Which is TRUE?**

- A. Friend function violates encapsulation
- B. Friend function is necessary sometimes
- C. Both
- D. None

**Answer: C**

---

**10. Friend function belongs to:**

- A. Object
- B. Class
- C. Namespace scope / global scope
- D. Constructor

**Answer: C**

---

**Section-2: Operator Overloading Basics**

**11. Operator overloading means:**

- A. Creating new operators
- B. Changing meaning of operator for user-defined types
- C. Modifying precedence
- D. Compiling operator as macro

**Answer: B**

---

**12. Which cannot be overloaded?**

- A. +
- B. ==
- C. ::
- D. []

**Answer: C**

---

**13. Overloaded operator function name format is:**

- A. operator+
- B. +operator
- C. overload+
- D. op+

**Answer: A**

---

**14. For binary operator as member function, how many arguments are passed explicitly?**

- A. 0
- B. 1
- C. 2
- D. 3

**Answer: B**

(Because left object is implicit via this)

---

**15. For unary operator as member function, arguments passed explicitly:**

- A. 0
- B. 1
- C. 2
- D. Depends

**Answer: A**

---

**16. When left operand is NOT an object (e.g., 25 + obj) — overloading should be:**

- A. Member function
- B. Friend/global function
- C. Static
- D. Inline

**Answer: B**

---

**17. Which operator requires two versions (pre & post)?**

- A. ++
- B. +
- C. ==
- D. []

**Answer: A**

---

**18. Post-increment operator signature contains:**

- A. No parameter
- B. One dummy int parameter
- C. Two parameters
- D. Reference parameter only

**Answer: B**

---

### **Section-3: Copy Constructor, Assignment Operator, Deep Copy**

**19. Explicit copy constructor is needed when:**

- A. Class has static members
- B. Class has DMA / pointer members
- C. Class has built-in variables
- D. Class has inline functions

**Answer: B**

---

**20. If DMA exists and default assignment operator is used:**

- A. Deep copy happens
- B. Shallow copy happens
- C. No copy happens
- D. Compile-time error

**Answer: B**

---

**21. Assignment operator must be overloaded when:**

- A. Pointer member exists
- B. Destructor deletes memory
- C. Copy constructor copies memory
- D. All above

**Answer: D**

---

**22. Correct order to avoid double delete issue:**

- A. Only destructor

- B. Destructor + copy constructor
- C. Destructor + copy constructor + operator=
- D. None

**Answer: C**

---

#### **Section-4: [], ->, \* Operator Overloading**

**23. Subscript operator must be overloaded as:**

- A. Friend function
- B. Member function
- C. Static function
- D. Global function

**Answer: B**

---

**24. Overloaded subscript returns:**

- A. Value only
- B. Reference when modification is needed
- C. Pointer only
- D. Object only

**Answer: B**

---

**25. Overloading operator-> allows:**

- A. Class object to behave like pointer
- B. Pointer to convert to class
- C. Inline execution
- D. Memory pooling

**Answer: A**

---

**26. operator overloaded to return reference enables:\***

- A. (obj) syntax
- B. obj\* syntax
- C. obj++ syntax
- D. Pointer cast

**Answer: A**

---

#### **Section-5: new & delete Overloading**

**27. operator new executes:**

- A. Before constructor
- B. After constructor
- C. After destructor
- D. At compile time

**Answer: A**

---

**28. operator delete executes:**

- A. Before destructor
- B. After destructor
- C. Before constructor
- D. Only on stack

**Answer: B**

---

**29. new/delete functions must return/use:**

- A. this pointer
- B. malloc/free internally

- C. static local storage
- D. constructor chaining

**Answer: B**

---

**30. Why overloaded new/delete must be static?**

- A. Memory reasons
- B. They are called without object
- C. Performance
- D. Access control

**Answer: B**

---

**Section-6: Smart Pointers**

**31. Smart pointers help avoid:**

- A. Memory leaks
- B. Dangling pointers
- C. Manual delete
- D. All above

**Answer: D**

---

**32. unique\_ptr supports:**

- A. Copy
- B. Assignment copy
- C. Move only
- D. None

**Answer: C**

---

**33. shared\_ptr destroys object when:**

- A. First owner released
- B. Last owner released
- C. Deleted manually only
- D. Compiler decides

**Answer: B**

---

**34. shared\_ptr uses:**

- A. Garbage collector
- B. Reference counting
- C. Stack unwinding
- D. OS cleanup

**Answer: B**

---

**Section-7: Type Conversion**

**35. Conversion direction meaning L = R refers to:**

- A. Left converted to right
- B. Right converted to left
- C. Both converted
- D. None

**Answer: B**

---

**36. Primitive → User-defined conversion can be done using:**

- A. Parameterized constructor
- B. Operator=
- C. Both

D. None

**Answer: C**

---

**37. User-defined → Primitive conversion uses:**

- A. Parameterized constructor
- B. Conversion operator
- C. Assignment operator
- D. Destructor

**Answer: B**

---

**38. Conversion operator function:**

- A. Has return type
- B. Has no return type keyword
- C. Takes parameters
- D. Static always

**Answer: B**

---

**39. Conversion operator must be:**

- A. Friend
- B. Member
- C. Static
- D. Virtual

**Answer: B**

---

**40. For UD → UD conversion, valid approaches are:**

- A. Conversion operator in source
- B. Operator= in destination
- C. Both
- D. None

**Answer: C**

---



## **Section-8: Output-Based MCQs**

---

**41.**

```
class A{
 int x;
public:
 A(int x){ this->x=x; }
 friend void disp(A a);
};
```

```
void disp(A a){
 cout<<a.x;
}
```

```
int main(){
 A obj(10);
 disp(obj);
}
```

Output?

- A. 0
- B. 10

C. Garbage

D. Error

**Answer: B**

---

**42.**

```
class A{
 int x=5;
public:
 int operator+(A &o){
 return x+o.x;
 }
};
```

```
int main(){
 A a,b;
 cout<<a+b;
}
```

A. Error

B. 5

C. 10

D. Undefined

**Answer: C**

---

**43.**

```
class A{
 int x;
public:
 A(int x){this->x=x;}
 A operator++(int){
 A t=*this;
 x++;
 return t;
 }
 void disp(){ cout<<x; }
};
```

```
int main(){
 A a(5);
 A b=a++;
 b.disp();
}
```

A. 5

B. 6

C. Error

D. Garbage

**Answer: A**

---

**44.**

```
class A{
 int x;
public:
 A(int x){this->x=x;}
```

```
A operator++(){
 ++x;
 return *this;
}
void disp(){ cout<<x; }
};
```

```
int main(){
 A a(5);
 A b=++a;
 b.disp();
}
```

A. 5  
B. 6  
C. Error  
D. Undefined

**Answer: B**

---

**45.**

```
class A{
 int x;
public:
 A(int x){this->x=x;}
 operator int(){
 return x;
 }
};
```

```
int main(){
 A a(10);
 cout<<a+5;
}
```

A. 10  
B. 5  
C. 15  
D. Error

**Answer: C**

---



**SECTION-1: BASICS & ACCESS RULES**

**1. Inheritance establishes which relationship?**

- A. Has-a
- B. Uses-a
- C. Is-a
- D. Contains-a

**Answer: C**

---

**2. Composition establishes which relationship?**

- A. Is-a
- B. Has-a
- C. Uses-a
- D. Owns-a

**Answer: B**

---

**3. Which members are NOT inherited in C++?**

- A. Private members
- B. Static members
- C. Constructors & destructor
- D. Public members

**Answer: C**

---

**4. In public inheritance, public members of base become:**

- A. Private
- B. Protected
- C. Public
- D. Static

**Answer: C**

---

**5. In protected inheritance, public members of base become:**

- A. Public
- B. Private
- C. Protected
- D. Static

**Answer: C**

---

**6. In private inheritance, protected base members become:**

- A. Public
- B. Private
- C. Protected
- D. None

**Answer: B**

---

**7. Private members of base class are:**

- A. Directly inherited & accessible
- B. Inherited but not directly accessible
- C. Not inherited at all
- D. Accessible via pointer

**Answer: B**

(They exist but are inaccessible directly)

---

**8. A protected member is accessible:**

- A. Only in base
- B. In derived & outside class
- C. In derived only
- D. Nowhere

**Answer: C**

---

**9. Which statement is TRUE?**

- A. Private and protected behave the same
- B. Private and protected differ only in inheritance
- C. Protected means global access
- D. Protected variables cannot be inherited

**Answer: B**

---

**10. Friendship is:**

- A. Inherited
- B. Not inherited
- C. Automatically cascaded
- D. Applied only to derived

**Answer: B**

---

## **SECTION-2: CONSTRUCTORS & DESTRUCTORS**

**11. Order of constructor calls in inheritance is:**

- A. Derived → Base
- B. Base → Derived
- C. Random
- D. Depends on compiler

**Answer: B**

---

**12. Order of destructor calls is:**

- A. Base → Derived
- B. Derived → Base
- C. Random
- D. Same as constructors

**Answer: B**

---

**13. If base has parameterized constructor and no default constructor:**

- A. Derived must call it explicitly
- B. Compiler will create default
- C. Object cannot be created
- D. Runtime error occurs

**Answer: A**

---

**14. Base constructor invoked in derived using:**

- A. super()
- B. Base::Base()
- C. Initializer list
- D. Constructor chaining

**Answer: C**

---

**15. Base constructor can be:**

- A. Public only
- B. Public or private only
- C. Public or protected or private
- D. Static only

**Answer: C**

---

**16. A protected constructor means:**

- A. Base cannot be instantiated directly
- B. Base instantiation always allowed
- C. Derived cannot be instantiated
- D. Compile-time error

**Answer: A**

---

### **SECTION-3: TYPES OF INHERITANCE**

**17. Derived → Base → Another Derived is:**

- A. Multiple
- B. Multilevel
- C. Hybrid
- D. Hierarchical

**Answer: B**

---

**18. Single base → multiple children is:**

- A. Multiple
- B. Multilevel
- C. Hybrid
- D. Hierarchical

**Answer: D**

---

**19. One class deriving from multiple bases is:**

- A. Hybrid
- B. Multiple
- C. Hierarchical
- D. Multilevel

**Answer: B**

---

**20. Combination of two or more inheritance types is:**

- A. Multilevel
- B. Hybrid
- C. Multiple
- D. Hierarchical

**Answer: B**

---

### **SECTION-4: MULTIPLE INHERITANCE RULES**

**21. Constructor call order in:**

class D : public B1, protected B2, private B3 {};

- A. D → B1 → B2 → B3
- B. B3 → B2 → B1 → D
- C. B1 → B2 → B3 → D
- D. Depends on access specifier

**Answer: C**

---

**22. Destructor call order is:**

- A. D → B1 → B2 → B3
- B. B3 → B2 → B1 → D
- C. D → B3 → B2 → B1
- D. Random

**Answer: C**

---

**23. If B1 & B2 have same method name, calling from D requires:**

- A. Ambiguity resolution
- B. Function overloading
- C. Multiple definition error
- D. Runtime error

**Answer: A**

---

**24. Ambiguity resolved using:**

- A. using keyword
- B. Namespace
- C. Scope resolution operator
- D. Virtual function

**Answer: C**

---

## **SECTION-5: VIRTUAL INHERITANCE & DIAMOND**

**25. Diamond problem arises due to:**

- A. Copy constructor
- B. Function hiding
- C. Multiple copies of base
- D. Default arguments

**Answer: C**

---

**26. Virtual inheritance ensures:**

- A. Base not inherited
- B. Single shared base instance
- C. Base treated as interface
- D. Destructor not called

**Answer: B**

---

**27. Which is correct to solve diamond problem?**

- A. class A : public B, public C
- B. class B : public A
- C. class B : virtual public A
- D. class A : virtual public B

**Answer: C**

---

**28. With virtual inheritance, base constructor executes:**

- A. Twice
- B. Once
- C. Never
- D. Compiler-dependent

**Answer: B**

---

**29. Destructor of virtual base executes:**

- A. First

- B. Last
- C. Random
- D. Depends

**Answer: B**

---

#### **SECTION-6: FRIEND + INHERITANCE**

**30. A friend of base is automatically friend of derived.**

- A. True
- B. False

**Answer: B**

---

#### **SECTION-7: FUNCTION HIDING / REDEFINITION**

**31. When derived defines function with same name & signature:**

- A. Overloading
- B. Overriding
- C. Redefinition (hiding)
- D. Polymorphism

**Answer: C**

---

**32. When function names match but parameters differ:**

- A. Overriding
- B. Hiding
- C. Overloading
- D. Virtual overriding

**Answer: B**

---

**33. Hidden base class function can be accessed using:**

- A. this pointer
- B. Friend function
- C. Base::function()
- D. virtual keyword

**Answer: C**

---

**34. To expose base overloads in derived, we use:**

- A. using Base::func;
- B. friend Base::func;
- C. public Base::func;
- D. Base::func();

**Answer: A**

---

#### **SECTION-8: COMPOSITION / AGGREGATION / ORDER**

**35. Composition implies:**

- A. Shared lifetime
- B. Independent lifetime
- C. No lifetime relation
- D. Stack lifetime only

**Answer: A**

---

**36. Aggregation implies:**

- A. Object owned & destroyed
- B. Object lifetime independent
- C. Always static

D. Raw pointer compulsory

**Answer: B**

---

**37. Constructor order when both inheritance & composition exist:**

- A. Derived → Base → Member
- B. Member → Base → Derived
- C. Base → Member → Derived
- D. Base → Derived → Member

**Answer: C**

---

**38. Destructor order for same case:**

- A. Derived → Member → Base
- B. Base → Member → Derived
- C. Member → Derived → Base
- D. Derived → Base → Member

**Answer: A**

---

## **SECTION-9: OUTPUT-BASED**

**39.**

```
class B{public: B(){cout<<"B ";}};
class D:public B{public:D(){cout<<"D ";}};
int main(){D d;}
```

Output:

- A. D B
- B. B D
- C. BD
- D. DB

**Answer: B**

---

**40.**

```
class A{public:A(){cout<<"A ";}};
class B:public A{public:B(){cout<<"B ";}};
class C:public B{public:C(){cout<<"C ";}};
int main(){C c;}
```

Output:

- A. C B A
- B. A B C
- C. B A C
- D. A C B

**Answer: B**

---

**41.**

```
class A{public:~A(){cout<<"A ";}};
class B:public A{public:~B(){cout<<"B ";}};
int main(){B b;}
```

Output:

- A. A B
- B. B A
- C. AB
- D. BA

**Answer: B**

---

**42.**

```
class A{public:void f(){cout<<"A";}};
class B:public A{public:void f(){cout<<"B";}};
int main(){B b;b.f();}
```

- A. A
- B. B
- C. Error
- D. Undefined

**Answer: B**

---

**43.**

```
class A{public:void f(int){cout<<"A";}};
class B:public A{public:void f(){cout<<"B";}};
int main(){B b;b.f(10);}
```

- A. A
- B. B
- C. Error
- D. Undefined

**Answer: C**

(Base overload hidden)

---

**44.**

```
class A{public:A(){cout<<"A";}};
class B:virtual public A{public:B(){cout<<"B";}};
class C:virtual public A{public:C(){cout<<"C";}};
class D:public B,public C{public:D(){cout<<"D";}};
int main(){D d;}
```

Output:

- A. ABCD
- B. A B C D
- C. A B C D A
- D. A B D C

**Answer: B**

---

#### SECTION-10: TRUE / FALSE

**45. Virtual inheritance removes duplicate base instances.**

- A. True
- B. False

**Answer: A**

---

**46. Multiple inheritance is not allowed in C++.**

- A. True
- B. False

**Answer: B**

---

**47. Base constructor must always be called explicitly.**

- A. True
- B. False

**Answer: B**

---

**48. Using-declaration can restore hidden base overloads.**

- A. True

B. False

**Answer: A**

---

**49. Friend function access is inherited automatically.**

A. True

B. False

**Answer: B**

---

**50. Composition ensures contained objects are destroyed with owner.**

A. True

B. False

**Answer: A**

---



## **SECTION-1 — UPCASTING & POINTER BEHAVIOR**

### **1. Upcasting means:**

- A. Base pointer to derived object
- B. Derived pointer to base object
- C. Derived object copied to base object
- D. Base object copied to derived object

**Answer: A**

---

### **2. Upcasting is generally allowed only under:**

- A. Private inheritance
- B. Protected inheritance
- C. Public inheritance
- D. Friend class

**Answer: C**

---

### **3. After upcasting, which functions are callable?**

- A. Only base class functions
- B. Only derived functions
- C. Both base and derived always
- D. None

**Answer: A**

---

### **4. If a function is NOT virtual, the function called depends on:**

- A. Object type
- B. Pointer/reference type
- C. Compiler setting
- D. RTTI

**Answer: B**

---

### **5. If a function IS virtual, the function called depends on:**

- A. Pointer type
- B. Reference type
- C. Object type
- D. Both object and pointer

**Answer: C**

---

## **SECTION-2 — BINDING**

### **6. Early binding happens when:**

- A. Function is virtual
- B. Function is non-virtual
- C. Call is via object
- D. Function is static

**Answer: B**

---

### **7. Late binding happens when:**

- A. Function is virtual and called through base pointer/reference
- B. Function is non-virtual
- C. Function is static
- D. Function is inline

**Answer: A**

---

**8. Which table correctly summarizes binding?**

- A. Virtual → compile-time
- B. Non-virtual → runtime
- C. Virtual via base pointer → runtime
- D. Virtual via object → runtime

**Answer: C**

---

### **SECTION-3 — REDEFINITION VS OVERRIDING**

**9. Overriding requires:**

- A. Same name only
- B. Same name & signature and base declared virtual
- C. Different parameters
- D. Friend declaration

**Answer: B**

---

**10. Redefinition (hiding) occurs when:**

- A. Base function is virtual
- B. Base function is static
- C. Function name matches but signature does not
- D. Access specifier differs

**Answer: C**

---

**11. In redefinition, which function executes?**

- A. Based on object
- B. Based on pointer/reference type
- C. Always base
- D. Always derived

**Answer: B**

---

### **SECTION-4 — RUNTIME POLYMORPHISM USE-CASES**

**12. Virtual functions work only when called through:**

- A. Object
- B. Base pointer/reference
- C. Derived object
- D. Friend function

**Answer: B**

---

**13. Array of base pointers holding derived objects demonstrates:**

- A. Compile-time polymorphism
- B. Runtime polymorphism
- C. Multiple inheritance
- D. Slicing

**Answer: B**

---

### **SECTION-5 — VTABLE / VPTR MECHANISM**

**14. VTABLE is created when:**

- A. Class has a constructor
- B. Class has static data
- C. Class contains at least one virtual function
- D. Class is abstract

**Answer: C**

---

---

**15. VPTR is stored:**

- A. In the class definition
- B. In every object
- C. In main() memory
- D. In static memory only

**Answer: B**

---

**16. VPTR is initialized by:**

- A. Destructor
- B. Compiler
- C. Programmer
- D. Friend function

**Answer: B**

---

**17. When virtual function is called:**

- A. Function jumps directly
- B. Function called via VTABLE lookup
- C. Static binding used
- D. Destructor runs

**Answer: B**

---

**18. Order of entries in VTABLE follows:**

- A. Function definition order
- B. Compiler choice
- C. Function usage order
- D. Random pattern

**Answer: A**

---

**SECTION-6 — MULTIPLE INHERITANCE WITH VTABLE**

**19. If two base classes have virtual functions, derived object contains:**

- A. One VPTR
- B. Two VPTRs
- C. No VPTR
- D. Depends on access specifier

**Answer: B**

---

**20. VPTR order in object layout depends on:**

- A. Alphabetical order
- B. Memory size
- C. Order of inheritance
- D. Function order

**Answer: C**

---

**SECTION-7 — VIRTUAL DESTRUCTOR**

**21. Why should base destructors be virtual?**

- A. To improve performance
- B. To enable multiple inheritance
- C. To destroy derived objects correctly via base pointer
- D. To avoid slicing

**Answer: C**

---

**22. If base destructor is NOT virtual and deleting derived via base pointer:**

- A. Both destructors run
- B. Only base destructor runs
- C. Only derived destructor runs
- D. Compile-time error

**Answer: B**

---

#### **SECTION-8 — VIRTUAL CALLS IN CONSTRUCTOR/DESTRUCTOR**

**23. Virtual calls inside constructors call:**

- A. Derived version
- B. Base version
- C. Undefined version
- D. Depends on RTTI

**Answer: B**

---

**24. Reason:**

- A. VPTR not created
- B. Only base part exists during construction
- C. Compiler ignores virtual
- D. Derived skipped

**Answer: B**

---

#### **SECTION-9 — ABSTRACT CLASS & PURE VIRTUAL**

**25. Abstract class is one that contains:**

- A. Static function
- B. Virtual destructor
- C. At least one pure virtual function
- D. Constructor only

**Answer: C**

---



**26. Abstract class can have:**

- A. Constructor
- B. Data members
- C. Non-virtual functions
- D. All of the above

**Answer: D**

---

**27. Abstract class cannot be:**

- A. Pointed to
- B. Referenced
- C. Instantiated
- D. Used in inheritance

**Answer: C**

---

#### **SECTION-10 — OBJECT SLICING**

**28. Object slicing occurs when:**

- A. Derived pointer assigned to base pointer
- B. Base object assigned derived object
- C. Reference assigned
- D. Virtual destructor omitted

**Answer: B**

---

**29. What gets copied in slicing?**

- A. Full derived object
- B. Only base part
- C. Only derived part
- D. None

**Answer: B**

---

**30. Slicing prevents:**

- A. Compilation
- B. Polymorphism
- C. Inheritance
- D. Function overloading

**Answer: B**

---

**SECTION-11 — OPERATOR INHERITANCE**

**31. operator= is NOT inherited because:**

- A. It is private
- B. It behaves like a constructor
- C. It must always be static
- D. Virtual behavior required

**Answer: B**

---

**32. operator+ is inherited and works on:**

- A. Entire derived object
- B. Only base part
- C. Derived part only
- D. Depends on virtual keyword

**Answer: B**

---



**SECTION-12 — COPY CONSTRUCTOR INHERITANCE**

**33. Compiler-generated copy constructor for derived:**

- A. Copies only base
- B. Calls base copy constructor & copies derived
- C. Does nothing
- D. Calls assignment operator

**Answer: B**

---

**34. If user defines copy constructor in derived but does NOT call base copy constructor:**

- A. Base copy still occurs
- B. Base default constructor runs
- C. Error
- D. Undefined

**Answer: B**

---

**SECTION-13 — OUTPUT QUESTIONS**

**35.**

```
class A{public:virtual void f(){cout<<"A";}};
class B:public A{public:void f(){cout<<"B";}};
int main(){A* p=new B;p->f();}
```

Output:

- A. A
- B. B

- C. Error
- D. Undefined

**Answer: B**

---

**36.**

```
class A{public:void f(){cout<<"A";}};
class B:public A{public:void f(){cout<<"B";}};
int main(){A* p=new B;p->f();}
```

Output:

- A. A
- B. B
- C. Error
- D. Depends

**Answer: A**

---

**37.**

```
class A{public:virtual ~A(){cout<<"A";}};
class B:public A{public::~B(){cout<<"B";}};
int main(){A* p=new B;delete p;}
```

Output:

- A. A
- B. B
- C. BA
- D. AB

**Answer: C**

---

**38.**

```
class A{public:A(){f();}virtual void f(){cout<<"A";}};
class B:public A{public:void f(){cout<<"B";}};
int main(){B b;}
```

Output:

- A. A
- B. B
- C. AB
- D. BA

**Answer: A**

---

**39.**

```
class A{public:virtual void f(){cout<<"A";}};
class B:public A{};
int main(){B b;A* p=&b;p->f();}
```

Output:

- A. A
- B. B
- C. Error
- D. Undefined

**Answer: A**

---

#### SECTION-14 — TRUE / FALSE

**40. Upcasting enables polymorphism.**

True

**Answer: True**

---

**41. Virtual constructor is allowed in C++.**

False (only factory pattern simulates it)

**Answer: False**

---

**42. Virtuality is inherited automatically.**

True

**Answer: True**

---

**43. Virtual calls inside destructor use late binding.**

False

**Answer: False**

---

**44. RTTI is stored in VTABLE.**

True

**Answer: True**

---

**45. Abstract class object cannot exist.**

False — pointer/reference allowed

**Answer: False**

---



## **SECTION-1 — BASICS OF EXCEPTION HANDLING**

**1. In C++, exceptions can be thrown as:**

- A. Only objects of exception class
- B. Only primitive types
- C. Primitive or user-defined objects
- D. Only `std::exception` types

**Answer: C**

---

**2. When you execute `throw e`; where `e` is an object:**

- A. Object is passed by reference
- B. Object is copied by value
- C. Object is moved
- D. A pointer is thrown

**Answer: B**

---

**3. Valid catch block syntax to catch any exception is:**

- A. `catch(any)`
- B. `catch(Exception)`
- C. `catch(...)`
- D. `catch(*)`

**Answer: C**

---

**4. Order of catch blocks must be:**

- A. Generic → Specific
- B. Specific → Generic
- C. Random
- D. Order does not matter

**Answer: B**

---

**5. If an exception is never caught, program ends using:**

- A. `abort()`
- B. `exit()`
- C. `std::terminate()`
- D. `std::exit()`

**Answer: C**

---

**6. Catching exception by reference (e.g., `catch(MyEx& e)`) is preferred because:**

- A. Avoids object slicing
- B. Avoids copying
- C. Supports polymorphism
- D. All the above

**Answer: D**

---

## **SECTION-2 — STACK UNWINDING & RAI**

**7. When an exception is thrown, destructors of local objects inside the try block execute in which order?**

- A. Same as construction
- B. Reverse of construction
- C. Unpredictable
- D. Not called

**Answer: B**

---

**8. This automatic cleanup process is called:**

- A. Garbage collection
- B. Stack rollback
- C. Stack unwinding
- D. Memory compression

**Answer: C**

---

**9. Stack unwinding destroys:**

- A. Only primitive types
- B. Only dynamically allocated objects
- C. Only global objects
- D. All automatic local objects in scope

**Answer: D**

---

### **SECTION-3 — `std::exception` HIERARCHY**

**10. `std::logic_error` represents:**

- A. Runtime environment error
- B. Hardware error
- C. Program logic error
- D. Input failure

**Answer: C**

---

**11. `std::runtime_error` is used when:**

- A. Logic is wrong
- B. Error occurs at program execution
- C. Code has syntax error
- D. Compiler fails

**Answer: B**

---

### **SECTION-4 — THROWING & CATCHING BEHAVIOR**

**12. Which catch matches `throw 10`;**

- A. `catch(double)`
- B. `catch(char)`
- C. `catch(int)`
- D. `catch(...)` only

**Answer: C**

---

**13. If no catch matches:**

- A. Program skips throw
- B. Undefined behavior
- C. `std::terminate()` is called
- D. Code returns to try

**Answer: C**

---

**14. Multiple matching catch blocks:**

- A. All are executed
- B. Only first matching block executes
- C. Last matching executes
- D. None execute

**Answer: B**

---

## SECTION-5 — RTTI BASICS

### 15. RTTI stands for:

- A. Runtime Table Type Info
- B. Runtime Type Identification
- C. Real Time Type Integration
- D. Run-Type Type Instance

**Answer: B**

---

### 16. RTTI is required mainly due to:

- A. Templates
- B. Operator overloading
- C. Runtime polymorphism
- D. Namespaces

**Answer: C**

---

### 17. RTTI provides:

- A. Compile-time type info
- B. Runtime type info
- C. Both
- D. None

**Answer: B**

---

## SECTION-6 — typeid OPERATOR

### 18. typeid(obj) returns:

- A. A string
- B. A type\_info object reference
- C. Object pointer
- D. Class name

**Answer: B**

---

### 19. Header needed for typeid:

- A. <rtti>
- B. <exception>
- C. <typeinfo>
- D. <cast>

**Answer: C**

---

*\*20. For NON-polymorphic classes, typeid(ptr) checks:*

- A. Actual object
- B. Pointer type
- C. Base class always
- D. Function type

**Answer: B**

---

*\*21. For POLYMORPHIC classes, typeid(ptr) checks:*

- A. Pointer type
- B. Compile-time type
- C. Actual object type
- D. None

**Answer: C**

---

## SECTION-7 — dynamic\_cast

**22. dynamic\_cast requires:**

- A. Templates
- B. Virtual function in base
- C. Friend function
- D. Public inheritance

**Answer: B**

---

**23. If dynamic\_cast to pointer fails, it returns:**

- A. NULL
- B. -1
- C. Default object
- D. Throws exception

**Answer: A**

---

**24. If dynamic\_cast to reference fails:**

- A. NULL reference
- B. nullptr
- C. bad\_cast exception
- D. Program crash

**Answer: C**

---

**25. dynamic\_cast is used mainly for:**

- A. Upcasting
- B. Downcasting safely
- C. Type conversion
- D. Void pointer conversion

**Answer: B**

---

**26. dynamic\_cast checks type using:**

- A. Compiler lookup
- B. RTTI via vtable
- C. Symbol table
- D. Garbage collector

**Answer: B**

---

## **SECTION-8 — static\_cast vs dynamic\_cast**

**27. static\_cast during downcasting:**

- A. Is runtime safe
- B. Uses RTTI
- C. Is unsafe if type mismatch
- D. Always fails

**Answer: C**

---

**28. dynamic\_cast works only with:**

- A. Primitive types
- B. Polymorphic classes
- C. Templates
- D. Static objects

**Answer: B**

---

**29. static\_cast for polymorphic downcast should be:**

- A. Always used

- B. Preferred over `dynamic_cast`
- C. Avoided unless guaranteed safe
- D. Required by compiler

**Answer: C**

---

#### **SECTION-9 — `const_cast`**

**30. `const_cast` is used to:**

- A. Change type
- B. Remove `const` / `volatile`
- C. Convert base to derived
- D. Modify pointer value

**Answer: B**

---

**31. Modifying a truly `const` variable via `const_cast` causes:**

- A. Compile-time error
- B. Safe update
- C. Undefined behavior
- D. Runtime error

**Answer: C**

---

#### **SECTION-10 — `reinterpret_cast`**

**32. `reinterpret_cast` is mainly for:**

- A. Safe numeric conversion
- B. Logical conversion
- C. Low-level bit reinterpretation
- D. Polymorphic casting

**Answer: C**

---

**33. `reinterpret_cast` is:**

- A. Type safe
- B. Checked cast
- C. Dangerous if misused
- D. Equivalent to `dynamic_cast`

**Answer: C**

---

#### **SECTION-11 — CO-VARIANT RETURN TYPE**

**34. Co-variant return type allows derived class to:**

- A. Change parameter list
- B. Return unrelated type
- C. Return derived type of base return
- D. Return primitive instead

**Answer: C**

---

**35. Co-variance applies only to:**

- A. Value returns
- B. Reference or pointer returns
- C. Friend functions
- D. Static functions

**Answer: B**

---

**36. Condition to allow co-variant return:**

- A. Base function must be inline

- B. Base function must be virtual
- C. Function must be static
- D. Derived must be abstract

**Answer: B**

---

**37. Which is INVALID?**

Base returns Child\*, derived returns Parent\*

- A. Valid
- B. Invalid
- C. Depends on compiler
- D. Works at runtime only

**Answer: B**

---

**SECTION-12 — OUTPUT-BASED**

**38.**

```
try{
 throw 5;
}
catch(double e){ cout<<"D"; }
catch(int e){ cout<<"I"; }
```

Output?

- A. Error
- B. D
- C. I
- D. None

**Answer: C**

---

**39.**

```
class A{ public: virtual void f(){cout<<"A";}};
class B:public A{ public: void f(){cout<<"B";}};
void test(A* p){ throw p; }
int main(){ B b; try{ test(&b);} catch(A* p){ p->f();}}
```

Output?

- A. A
- B. B
- C. Error
- D. Undefined

**Answer: B**

---

**40.**

```
class A{};
int main(){
 try{ throw A(); }
 catch(...){ cout<<"X"; }
}
```

Output?

- A. Compile error
- B. Runtime error
- C. X
- D. None

**Answer: C**

---

**41.**

```
class Base{ virtual void f(){} };
class D:public Base{};
Base* b=new Base;
cout<<(typeid(*b)==typeid(D));
Output?
```

- A. 0
- B. 1
- C. Error
- D. Undefined

**Answer: A**

---

**42.**

```
class B{ virtual void f(){} };
class D:public B{};
B* b=new D;
cout<<(typeid(*b)==typeid(D));
Output?
```

- A. 0
- B. 1
- C. Error
- D. Undefined

**Answer: B**

---

**43.**

```
class B{ virtual void f(){} };
class D:public B{};
B* b=new B;
D* d=dynamic_cast<D*>(b);
cout<<(d==NULL);
Output?
```

- A. 0
- B. 1
- C. Error
- D. Undefined

**Answer: B**

---

**44.**

```
class B{};
class D:public B{};
B* b=new B;
D* d=dynamic_cast<D*>(b);
A. Compiles, d becomes NULL
B. Runtime error
C. Compile error
D. Undefined
```

**Answer: C**

(Because base is NOT polymorphic)

---

## SECTION-13 — TRUE / FALSE

**45. C++ can throw primitive values as exceptions.**

True

**Answer: True**

---

**46. Stack unwinding destroys objects allocated via new.**

False — only automatic objects are destroyed

**Answer: False**

---

**47. dynamic\_cast uses RTTI internally.**

True

**Answer: True**

---

**48. Co-variant return types improve type safety.**

True

**Answer: True**

---

**49. static\_cast performs runtime checking.**

False

**Answer: False**

---

**50. typeid works only on polymorphic classes.**

False — works for all, but behavior differs

**Answer: False**

---



### SECTION-1 — STREAM BASICS & HIERARCHY

1. A stream in C++ refers to:

- A. A file stored on disk
- B. A communication path between source and destination
- C. A memory buffer
- D. A network channel

**Answer: B**

---

2. Which of the following is NOT a C++ stream class?

- A. istream
- B. ostream
- C. ifstream
- D. file\_stream

**Answer: D**

---

3. cin is an object of:

- A. ostream
- B. istream
- C. iostream
- D. fstream

**Answer: B**

---

4. cout is an object of:

- A. ostream
- B. istream
- C. fstream
- D. iostream

**Answer: A**

---

5. The base class of all stream classes in C++ is:

- A. stream
- B. ios
- C. base\_stream
- D. filebase

**Answer: B**

---

### SECTION-2 — FILE STREAM CLASSES

6. Which class is used to read from files?

- A. ofstream
- B. ifstream
- C. fstream
- D. iostream

**Answer: B**

---

7. Which class is used to write to files?

- A. ostream
- B. ifstream
- C. ofstream
- D. iostream

**Answer: C**

---

**8. Which class supports both reading and writing?**

- A. ostream
- B. fstream
- C. ifstream
- D. io\_base

**Answer: B**

---

**9. Header file required for file streams is:**

- A. <iostream>
- B. <file>
- C. <fstream>
- D. <stdio.h>

**Answer: C**

---

### **SECTION-3 — OPENING & CLOSING FILES**

**10. Default open mode of ifstream is:**

- A. ios::out
- B. ios::in
- C. ios::app
- D. ios::binary

**Answer: B**

---

**11. Default open mode of ofstream is:**

- A. ios::out
- B. ios::out | ios::trunc
- C. ios::app
- D. ios::binary

**Answer: B**

---

**12. fstream default mode is:**

- A. ios::out
- B. ios::in
- C. ios::in | ios::out
- D. ios::app

**Answer: C**

---

**13. To close a file stream we use:**

- A. stop()
- B. shutdown()
- C. end()
- D. close()

**Answer: D**

---

### **SECTION-4 — FILE OPEN STATUS**

**14. Which function checks file open failure?**

- A. bad()
- B. error()
- C. fail()
- D. wrong()

**Answer: C**

---

**15. Which operator can be used to test open failure?**

- A. ==
- B. !=
- C. !
- D. &

**Answer: C**

---

**16. To check if a file is open:**

- A. is\_open()
- B. open\_status()
- C. file\_ok()
- D. ready()

**Answer: A**

---

## **SECTION-5 — BASIC FILE I/O**

**17. put() is used to:**

- A. Read a character
- B. Write a character
- C. Read a line
- D. Write a line

**Answer: B**

---

**18. get(char&) is used to:**

- A. Write a character
- B. Write a string
- C. Read a character
- D. Read a string

**Answer: C**

---

**19. getline() reads:**

- A. A character
- B. A token
- C. A full line
- D. A file

**Answer: C**

---

## **SECTION-6 — BINARY FILE OPERATIONS**

**20. Binary writing uses:**

- A. write()
- B. put()
- C. send()
- D. store()

**Answer: A**

---

**21. Binary reading uses:**

- A. read()
- B. get()
- C. load()
- D. fread()

**Answer: A**

---

**22. Bytes read by last read operation are obtained using:**

- A. bytes()
- B. length()
- C. gcount()
- D. size()

**Answer: C**

---

**23. Serialization means:**

- A. Converting binary to text
- B. Saving object to file
- C. Calling constructor
- D. File compression

**Answer: B**

---

## **SECTION-7 — FILE OPEN MODES**

**24. ios::app ensures:**

- A. Writing at beginning
- B. Overwriting
- C. Writing only at end
- D. Binary mode

**Answer: C**

---

**25. ios::ate means:**

- A. Append only
- B. Open at end but allow writing anywhere
- C. Binary mode
- D. Truncate file

**Answer: B**

---

**26. ios::trunc means:**

- A. Append file
- B. Keep previous data
- C. Always clear existing data
- D. Open file in binary

**Answer: C**

---

## **SECTION-8 — RANDOM ACCESS**

**27. Function to move get pointer:**

- A. moveg()
- B. seekg()
- C. goto()
- D. jumpg()

**Answer: B**

---

**28. Function to move put pointer:**

- A. movep()
- B. seekp()
- C. outp()
- D. jump()

**Answer: B**

---

**29. tellg() returns:**

- A. File size
- B. Pointer address
- C. Current read position
- D. Current write position

**Answer: C**

---

**30. tellp() returns:**

- A. Current read position
- B. File size
- C. File descriptor
- D. Current write position

**Answer: D**

---

## **SECTION-9 — TEXT vs BINARY**

**31. In text mode, newline converts to:**

- A. \n
- B. \r
- C. \r\n
- D. none

**Answer: C**

---

**32. Binary mode performs:**

- A. Newline translation
- B. EOF conversion
- C. No translation
- D. Compression

**Answer: C**

---

**33. Float 123.45 stored in binary mode uses:**

- A. 3 bytes
- B. 5 bytes
- C. 8 bytes
- D. 4 bytes

**Answer: D**

---

## **SECTION-10 — STREAM STATE FLAGS**

**34. eofbit is set when:**

- A. File open fails
- B. Write fails
- C. End of file reached
- D. File closed

**Answer: C**

---

**35. To reuse stream after EOF:**

- A. reset()
- B. restore()
- C. clear()
- D. reopen()

**Answer: C**

---

## **SECTION-11 — MANIPULATORS**

**36. setw(10) affects:**

- A. All output
- B. Only next output
- C. Entire program
- D. Numeric only

**Answer: B**

---

**37. fill('\*') means:**

- A. Replace spaces with \*
- B. Prefix with \*
- C. Add suffix \*
- D. None

**Answer: A**

---

**38. showpos displays:**

- A. Only minus sign
- B. Always + for positive numbers
- C. Nothing
- D. Zeros

**Answer: B**

---

**39. setprecision(3) with fixed means:**

- A. 3 significant digits
- B. 3 decimal places
- C. 3 hex digits
- D. Rounded to nearest integer

**Answer: B**

---

**40. boolalpha prints true as:**

- A. 1
- B. t
- C. yes
- D. true

**Answer: D**

---

**41. Manipulator to show hexadecimal is:**

- A. base(16)
- B. set(16)
- C. hex
- D. convert\_hex

**Answer: C**

---

**42. endl does:**

- A. Newline only
- B. Flush only
- C. Newline + flush
- D. None

**Answer: C**

---

**43. scientific output for 1234.5 is:**

- A. 1234.5
- B. 1.2345e+03

C. 123.45e+1

D. 1.2345

**Answer: B**

---

## SECTION-12 — ALIGNMENT

**44. ios::left causes:**

A. Right alignment

B. Left alignment

C. Center alignment

D. No effect

**Answer: B**

---

**45. ios::internal aligns:**

A. Sign left, number right

B. Both right

C. Both left

D. No alignment

**Answer: A**

---

## SECTION-13 — OUTPUT-BASED

**46.**

`cout << hex << 255;`

Output:

A. 255

B. ff

C. 0xff

D. F F

**Answer: B**

---

**47.**

`cout << setw(6) << 123;`

Output width is:

A. 3

B. 4

C. 6

D. Unknown

**Answer: C**

---

**48.**

`cout << fixed << setprecision(2) << 1.234;`

Output:

A. 1.2

B. 1.23

C. 1.234

D. 1.23000

**Answer: B**

---

**49.**

`cout << boolalpha << (10>5);`

Output:

A. 1

B. 0

C. true

D. false

**Answer: C**

---

**50.**

```
cout << setfill('*') << setw(6) << 25;
```

Output:

A. 25\*\*\*\*

B. \*\*\*\*25

C. \*\*25

D. 25

**Answer: B**

---



## ✔ PART-1 — NAMESPACES IN C++ MCQs

---

**1. A namespace is primarily used to:**

- A. Reduce memory usage
- B. Improve performance
- C. Prevent name conflicts
- D. Control program flow

**Answer: C**

---

**2. Which keyword is used to declare a namespace?**

- A. package
- B. module
- C. group
- D. namespace

**Answer: D**

---

**3. A namespace can contain:**

- A. Only variables
- B. Only functions
- C. Only classes
- D. Variables, functions, classes, enums, structs

**Answer: D**

---

**4. Which is the correct way to access a namespace member?**

- A. namespace.member
- B. namespace->member
- C. namespace::member
- D. namespace.member()

**Answer: C**

---

**5. A namespace can be declared:**

- A. Only once
- B. Multiple times in the same program
- C. Only inside classes
- D. Only inside functions

**Answer: B**

---

**6. Namespaces that have no name are called:**

- A. Hidden namespaces
- B. Anonymous namespaces
- C. Private namespaces
- D. Local namespaces

**Answer: B**

---

**7. Anonymous namespaces provide:**

- A. Public linkage
- B. Private linkage
- C. Internal linkage
- D. Protected linkage

**Answer: C**

---

**8. Which is equivalent to global static declaration?**

- A. Local namespace
- B. Inline namespace
- C. Anonymous namespace
- D. std namespace

**Answer: C**

---

**9. Which brings all names from a namespace into current scope?**

- A. using namespace ns;
- B. use ns.\*;
- C. import ns;
- D. load namespace ns;

**Answer: A**

---

**10. What is the disadvantage of using namespace std; globally?**

- A. Slows execution
- B. Causes memory leak
- C. Causes name conflicts
- D. Generates warnings

**Answer: C**

---

**11. Nested namespaces since C++17 can be written as:**

- A. namespace a(b)(c)
- B. namespace a.b.c
- C. namespace a::b::c
- D. namespace a/b/c

**Answer: C**

---

**12. Namespace aliasing syntax is:**

- A. alias ns = namespace::test;
- B. short ns = namespace test;
- C. namespace ns = test;
- D. name ns test;

**Answer: C**

---

**13. Functions inside namespaces are accessed through:**

- A. Scope resolution operator
- B. Dot operator
- C. Arrow operator
- D. None

**Answer: A**

---

**14. Inline namespaces are mainly useful for:**

- A. Security
- B. Versioning APIs
- C. Performance
- D. Exception handling

**Answer: B**

---

**15. Which feature finds functions based on argument type namespace?**

- A. RTTI
- B. ADL
- C. ODR

D. SFINAE

**Answer: B**

---

**16. Where should using namespace std; generally NOT be placed?**

- A. Inside main
- B. Inside function
- C. Inside namespace
- D. In header file

**Answer: D**

---

**17. The std namespace contains:**

- A. Only cout
- B. Only data structures
- C. All C++ Standard Library
- D. Only iostream

**Answer: C**

---

**18. Two namespaces with same name declared in different files become:**

- A. Separate
- B. Conflicting
- C. Same namespace
- D. Undefined

**Answer: C**

---

 **PART-2 — STL (STANDARD TEMPLATE LIBRARY) MCQs**

---

**19. STL consists of:**

- A. Containers only
- B. Algorithms only
- C. Iterators only
- D. Containers, Iterators, Algorithms

**Answer: D**

---

**20. Vector is implemented as:**

- A. Linked list
- B. Static array
- C. Dynamic array
- D. Binary tree

**Answer: C**

---

**21. Which supports random access?**

- A. list
- B. vector
- C. stack
- D. queue

**Answer: B**

---

**22. Which does NOT support random indexing?**

- A. vector
- B. deque
- C. list

D. string

**Answer: C**

---

**23. vector::push\_back()**

- A. Inserts at beginning
- B. Inserts at end
- C. Deletes element
- D. Clears vector

**Answer: B**

---

**24. Complexity of inserting at end of vector (amortized)?**

- A.  $O(1)$
- B.  $O(n)$
- C.  $O(\log n)$
- D.  $O(n \log n)$

**Answer: A**

---

**25. list is implemented as:**

- A. Circular array
- B. Doubly linked list
- C. Singly linked list
- D. Binary tree

**Answer: B**

---

**26. set stores elements as:**

- A. Unsorted
- B. Sorted unique
- C. Hash table
- D. Sequential list

**Answer: B**

---

**27. set is internally implemented as:**

- A. AVL tree
- B. B-Tree
- C. Red-Black tree
- D. Skip list

**Answer: C**

---

**28. map stores:**

- A. Only keys
- B. Only values
- C. Key-Value pairs
- D. Indexed array

**Answer: C**

---

**29. Stack follows:**

- A. FIFO
- B. LILO
- C. LIFO
- D. Random

**Answer: C**

---

**30. Which returns top element of a stack?**

- A. peek()
- B. top()
- C. head()
- D. get()

**Answer: B**

---

**31. Which string class is safer than char arrays?**

- A. CString
- B. string
- C. wstring
- D. std\_text

**Answer: B**

---

**32. Iterators behave like:**

- A. Arrays
- B. Objects
- C. Pointers
- D. Functions

**Answer: C**

---

**33. Which algorithm reverses a container?**

- A. flip()
- B. swap()
- C. reverse()
- D. invert()

**Answer: C**

---



### **PART-3 — TEMPLATES MCQs**

---

**34. Templates allow:**

- A. Writing one function for many types
- B. Faster execution
- C. More memory
- D. Better graphics

**Answer: A**

---

**35. Syntax keyword for template?**

- A. template<>
- B. typename
- C. class
- D. All of these

**Answer: D**

---

**36. Function templates are instantiated when:**

- A. Declared
- B. Defined
- C. First used
- D. Program ends

**Answer: C**

---

**37. Compiler generates code for template:**

- A. Once
- B. Per type used
- C. Per call
- D. Per program

**Answer: B**

---

**38. Overload resolution prefers:**

- A. Template function
- B. Non-template exact match
- C. Template specialization
- D. Any function

**Answer: B**

---

**39. Class template specialization is used when:**

- A. Overloading allowed
- B. Overloading disallowed
- C. Need special behavior for type
- D. Avoid inheritance

**Answer: C**

---

**40. Templates are best defined in:**

- A. .cpp
- B. .h
- C. .obj
- D. .lib

**Answer: B**

---

**41. If template definition is not visible during instantiation, the result is:**

- A. Compile error
- B. Runtime error
- C. Linker error
- D. Warning only

**Answer: C**

---

**42. Reason templates cannot be distributed as compiled libraries?**

- A. Compiler cannot know in advance which types will be used
- B. Too large
- C. Illegal in C++
- D. Slow

**Answer: A**

---

## **PART-4 — VARIADIC TEMPLATES & FOLD EXPRESSIONS**

---

**43. Variadic templates allow:**

- A. Only one argument
- B. Fixed arguments
- C. Variable number of arguments
- D. Compile-time only

**Answer: C**

---

**44. Parameter pack is declared as:**

- A. T... args
- B. args T...
- C. pack
- D. var

**Answer: A**

---

**45. sizeof...(args) gives:**

- A. Memory size
- B. Argument count
- C. Type count only
- D. Always 1

**Answer: B**

---

**46. Base case in recursive variadic function must be:**

- A. void disp(int)
- B. void disp(...)
- C. void disp()
- D. disp must not exist

**Answer: C**

---

**47. Fold expressions were introduced in:**

- A. C++03
- B. C++11
- C. C++14
- D. C++17

**Answer: D**

---

**48. Example of fold expression:**

- A. (args \*\*)
- B. (... + args)
- C. add(args...)
- D. fold(args)

**Answer: B**

---

**49. initializer\_list works when:**

- A. Arguments are mixed types
- B. Arguments are same type
- C. No arguments
- D. Only template

**Answer: B**

---

**50. Which prints all arguments?**

- A. print(args)
- B. cout << args;
- C. ((cout << args << " "), ...);
- D. display args...

**Answer: C**

---